time and
ficient the
shortcuts
rocess.
n dozen
erarchial

Date 13/2/26

UNIT-V

Graph Matrics and Application.

→ The graphical representation of another nodes (the states) is known as graph matrix

→ The matrix are used when the progra -mmer code are encrypts a message.

→ Matrix are also used in Computer animation.

→ Matrix are used with Computing in architecture

→ Math reports are recorded with Matrix in engineering

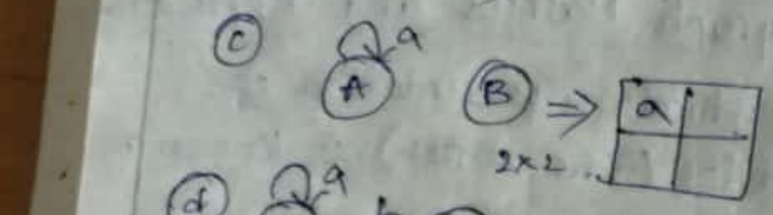
Matrix of a graph

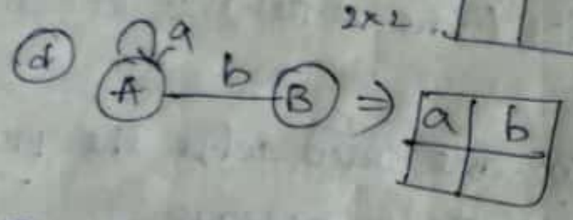
The another notation for representing graphs is known as graph matrix. Each and every node in the graph is represented a single row and a single column

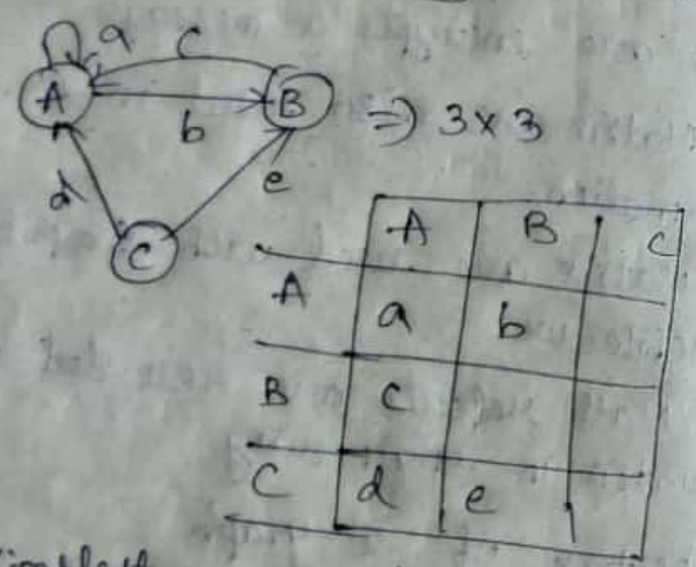
→ Some examples of graph and their matrix are given below.

a) (A) ⟹ [a]

(b)  ⇒ [a]

(c) 

(d) 

(e) 

The simplest weight to represent whether the connection exist or does not exist between the two nodes is the binary weights.

→ It can be represented as 1 and 0

The principles

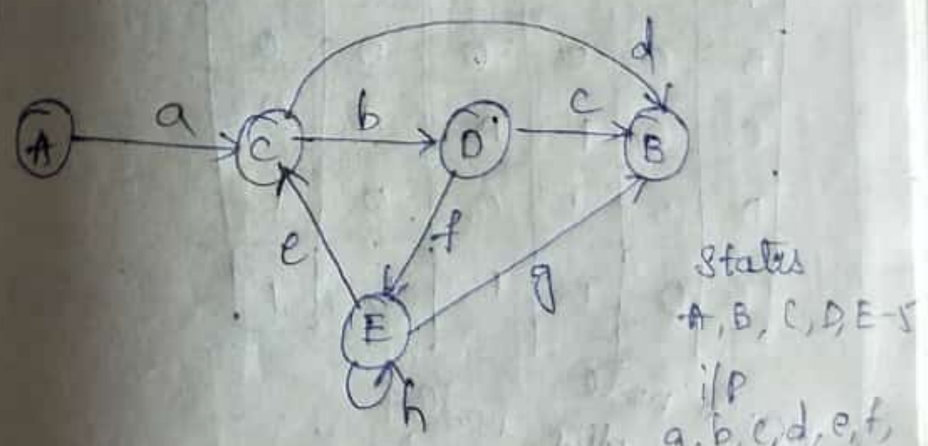
The size of the matrix number of rows 'x' no. of columns should be equal to the no. of nodes.

→ The direct link are connection between any two nodes can be put in the matrix format

The example of Connected Matrix

If the matrix is represented by the weights which are defined as above are known as connected matrix so that it can be represented a matrix with zero and one's is known as connected matrix.

The links of the state graph are represented as one and if there is no link it is represented as zero.



States
A, B, C, D, E - 5

i/p
a, b, c, d, e, f, g, h

Convert it into Matrix and connected matrix.

5x5 Matrix

	A	B	C	D	E
A			a		
B				.	
C		d		b	
D		C			e
E		g	e		h

Connected Matrix ; (P) → 1
remaing → 0.

	A	B	C	D	E
A	0	0	1	0	0
B	0	0	0	0	0
C	0	1	0	1	0
D	0	1	0	0	1
E	0	1	1	0	1

Branch node
Junction node
loop node

Date 16/3/26

Node Reduction Algorithm :- It is a combination of node reduction procedure and graph matrix.

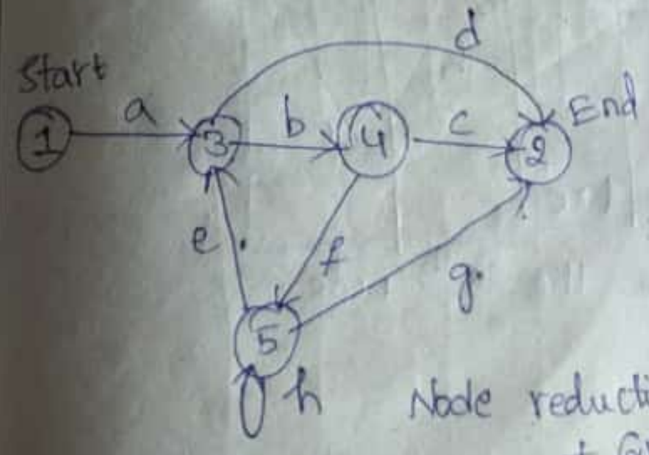
→ Node reduction procedure means we have to remove all the intermediate nodes between the start point and ending point

→ Identifying starting point and ending point.

→ Whenever we are removing intermediate nodes we are following 3 rules.

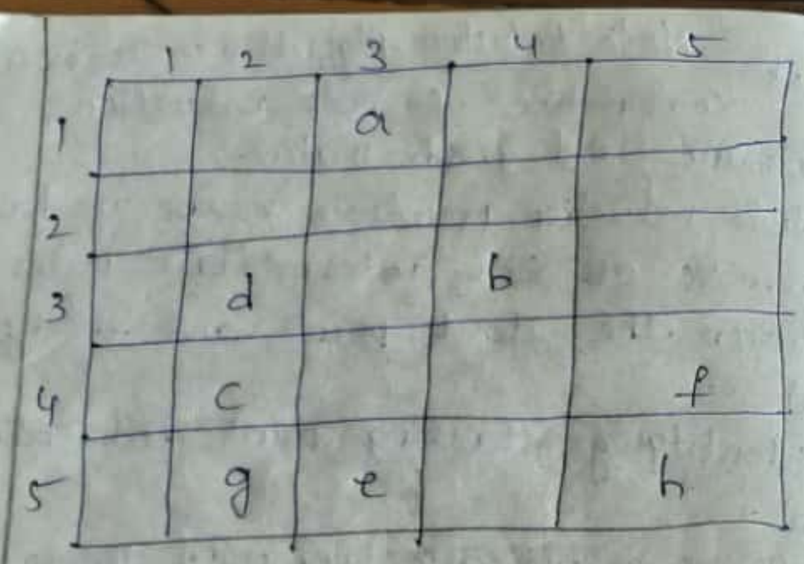
1. The links consisting in a series manner, we can multiply those links
2. The links consisting in a parallel we can add those links
3. The links consisting of loop we can use the closure

Example for the node Reduction Algorithm

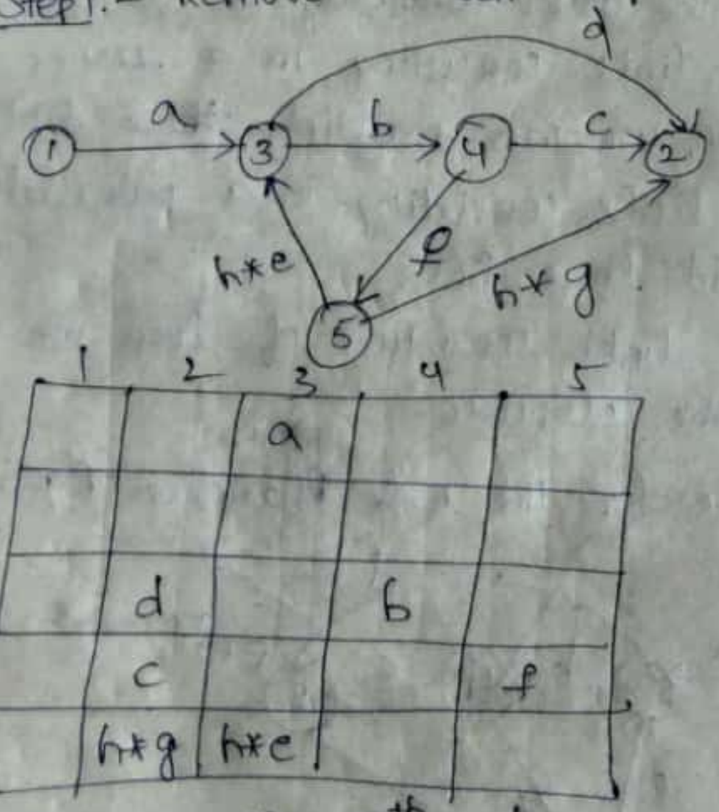


5 × 5

Node reduction procedure + Graph Matrix



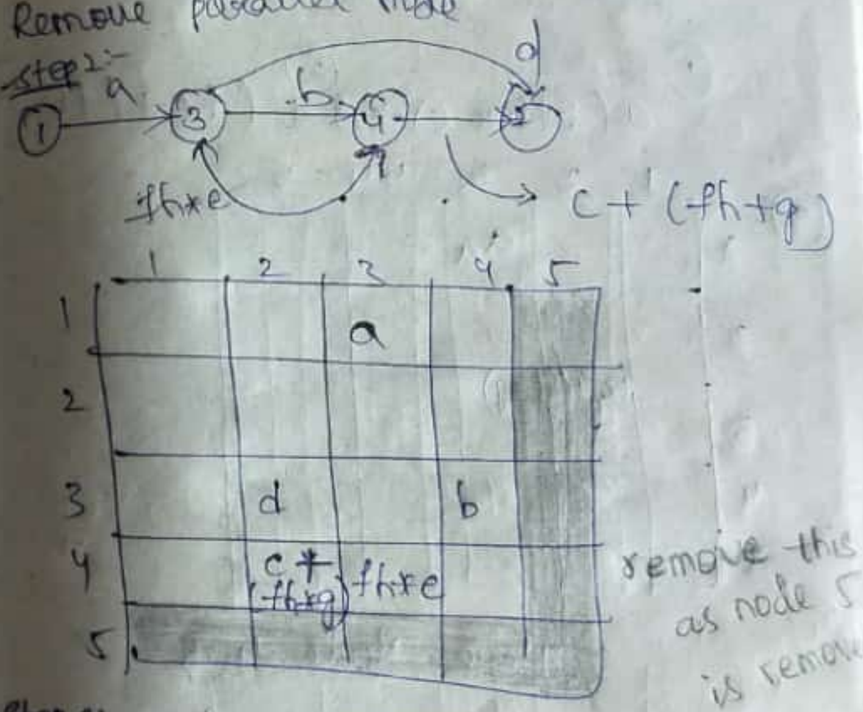
Step 1:- Remove the self loop at node 5



Remove the 5th node.

Remove parallel node

Step 2:-



Step 3:-

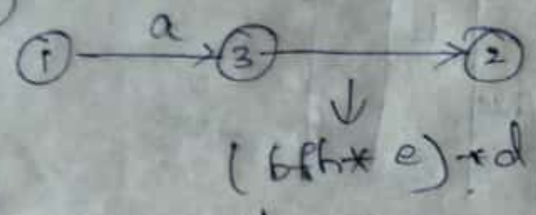


④

self loop at node 3

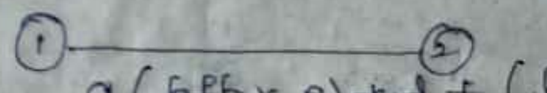
remove it

⑤



⑥

Remove 3rd node.



③

	1	2	3	4	5
1			a		
2					
3			bc+(fh+g) d		
4					
5					

Here Node 4, 5 are removed.

④

	1	2	3	4	5
1			a		
2					
3		d+(bc+(fh*g))bfhxe			
4					
5					

⑤

	1	2	3
1			a
2			
3		(bfh*e)*d+(bc+(fh*g))	

⑥

	1	2
1		a(bfh*e)*d+(bc+(fh*g))
2		

Graph Matrices and Applications

Motivational overview

The pictorial graphs are represented using another notation is known as Graph matrics.

→ The concept of graph matrices have been evolved inorder to solve the problems of pictorial graphs.

→ Graph matrices are used for proving the theorems.

→ The graph theory contains basic tool kit of three algorithms

* Matrix multiplication
* Partitioning Algorithm
* Matrix Determinant.

Applications of Graph Matrices

Matrices are used often the programmers codes or encrypts a message.

→ Geologists make use of matrices for seismic surveys.

→ Matrices are also used in computer Animation.

→ Matrices are used with computing in Architecture.

→ Math reports are recorded using matrices in Engineering.

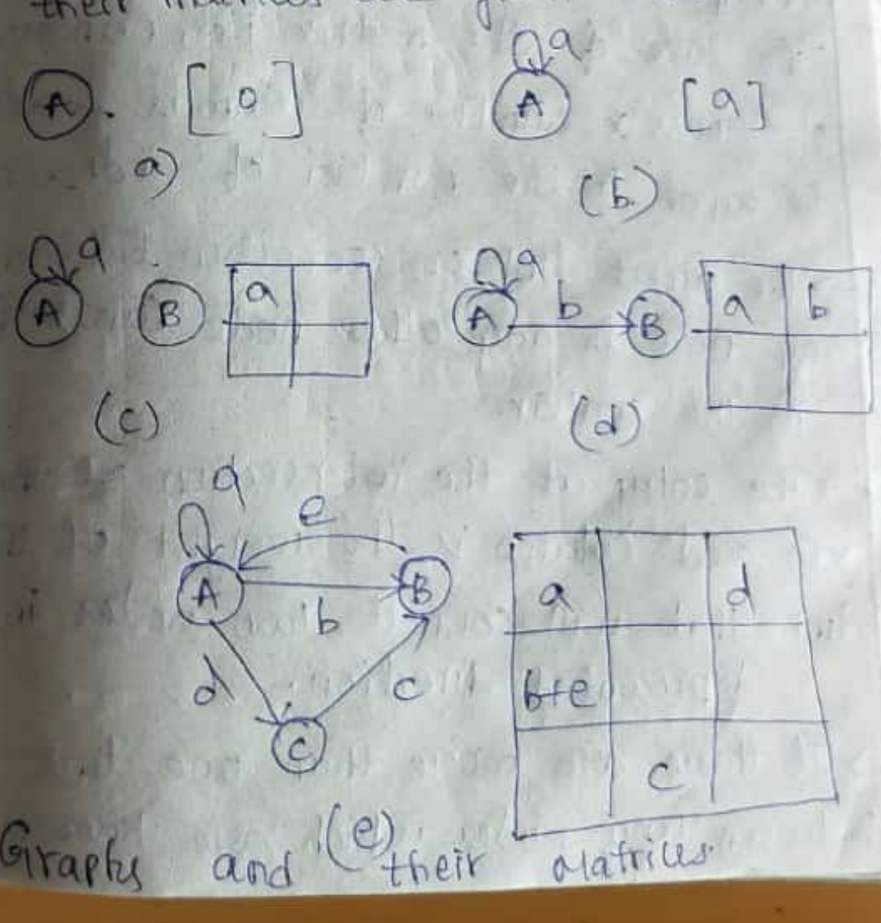
→ Matrices are used to calculate gross domestic product in economical goods can be produced more efficiently.

Matrix of a graph

The another notation for representing graphs is known as "Graph Matrix"

→ Each and every node in the graph is represented as a single row and a single column.

Examples:- some examples of graph and their matrices are given below.



Graphs and their Matrices

The simplest weight to represent the connection exists or does not exist between two nodes is the binary weights i.e., it is represented as "1" if link exists otherwise it is represent as "0"

→ The arithmetic rules for binary weights are,

1+1=1	1×1=1
1+0=1	1×0=1
0+0=0	0×0=0

Principles

The size of the matrix i.e., number of rows x number of columns should be equal to the number of nodes.

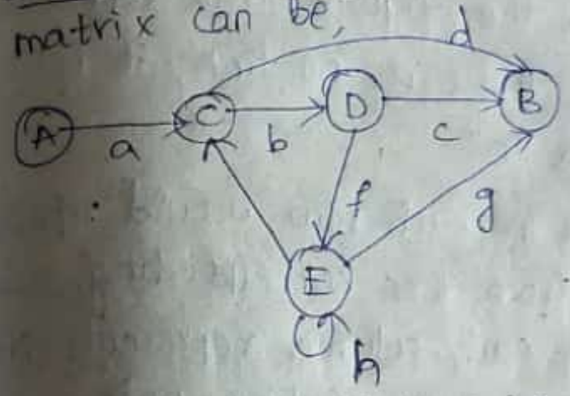
→ The direct link or connection between any node to any other node can be put in the matrix.

→ The entry at the intersection of the row and column is the weight of the link that will connect two nodes in represented direction.

→ If there are more than one link between two nodes which are known as parallel links, then the entry of those links in the matrix would be sum of all the link weight links.

→ If the matrix is represented by the weights which are defined as above are known as connected matrix i.e., representing a matrix with 0's and 1's known as connected matrix.

Example:- The example of a connected matrix can be,



The general matrix i.e., relational matrix and connection matrix of the Fig 8.2 can be represented as

	A	B	C	D	E
A			a		
B					
C		d		b	
D		c			f
E		g	e		h

	A	B	C	D	E
A	0	0	1	0	0
B	0	0	0	0	0
C	0	1	0	1	0
D	0	1	0	0	1
E	0	1	1	0	1

Fig 8.3 Matrix Representations.

The links of the figure are represented as "1" and if there is no link, it is represented as "0"

→ The entries in each row would represent the outlinks from the corresponding node and entries in each column represents the inlinks from the corresponding node.

→ The different nodes such as Branch node, Junction node, loop node in connection graph can be identified as previous.

Branch Node

If the row contains two or more 1's then it is a branch node. The rows C, D, E in the above figure as branch node.

→ The number of decisions at that particular node can be calculated by using n-1 formula.

So for the node

C → 2-1 = 1

D → 2-1 = 1

E → 2-1 = 1

Junction Node

→ If the column contains two or more 1's then it is a junction node.

→ The columns B, C, E in the above figure are known as "junction nodes".

Loop Node

If there is an entry "1" in the diagonal of corresponding node then that node is a loop node.

→ The node E in the figure is known as "Loop node"

Relations

→ A relation is nothing but a property that connects any two things.

→ If a and b are any things connected by Relation R so it is said that a has

relation R to b.

→ It is symbolically represented as aRb

properities of Relations

Transitive
Reflexive
Symmetric
Anti Symmetric

Transitive Relations :- If there exists two relations aRb and bRc and the relation R is said to have transitive property if :

aRb and bRc ⇒ aRc

Ex:- if a=b and b=c then a=c
if a>b and b>c then a>c

Reflexive Relations

If there exists two relations between a and a i.e., aRa then R is said to be Reflexive Relation.

Ex:- a=a (reflexive)
a=a (irreflexive)

Symmetric Relation:-

A relation R is said to symmetric if aRb implies bRa.

→ Symmetric Relation implies that if there is a link or connection between two nodes i.e, from a to b then there should also be a link from b to a.

→ The relations which are not symmetric would represent a "Directed Graph."

→ The graph with symmetric relations is known as "undirected Graph"

Ex:- if a=b then b=a (Symmetric)
if a>b then b>a (not Symmetric)

Anti Symmetric Relation

If the relation aRb and bRa implies that a=b then R is said to be a AntiSymmetric relations.

Ex:- if a=b and b=a then a=b
if a>b and b>a then a=b

Equivalence Relation:-

The relation which satisfies three properties i.e., Reflexive, symmetric and transitive are known as "Equivalence Relation"

Partitioning Algorithm

Consider any graph over a transitive relation. The graph may have loops.

→ We would like to partition the graph by grouping nodes in such a way that every loop is contained within one group or another.

Such a graph is partially ordered.

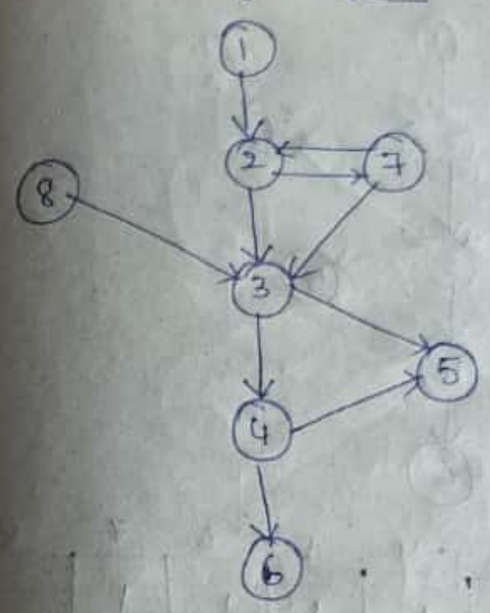
→ There are many used for an algorithm that does that.

→ We might want to embed the loops within a subroutine so as to have a resulting graph which is loop free at the top level.

→ Many graphs with loops are easy to analyze if you know where to break the loops.

→ While you and I can recognize loops, it's much harder to program a tool to do it unless you have a solid algorithm on which the tool is based.

Partitioning Algorithm



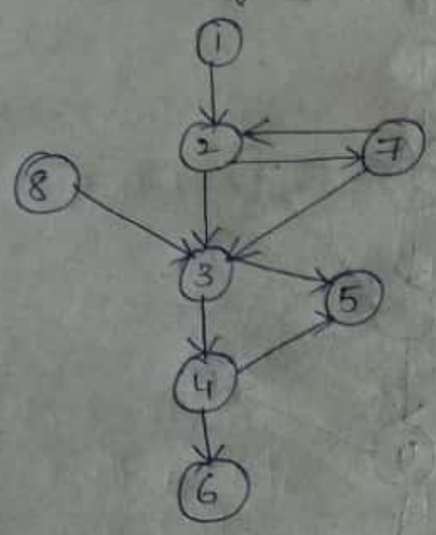
	1	2	3	4	5	6	7	8
1	1	1						
2		1	1				1	
3			1	1				
4				1	1	1		
5			1		1			
6						1		
7		1	1				1	
8			1					1

Relation Matrix

... Considering that the graph may have loops.

Diagonal entries are made to represent self loop.

Partitioning algorithm (cont---):

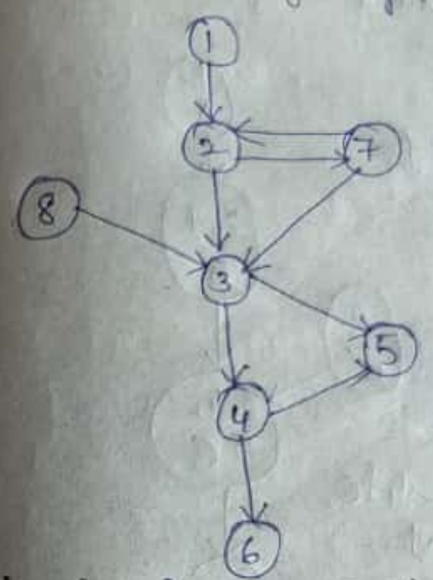


	1	2	3	4	5	6	7	8
1	1	1	1	1	1	1	1	
2		1	1	1	1	1	1	
3			1	1	1	1		
4			1	1	1	1		
5			1	1	1	1		
6						1		
7		1	1	1	1	1	1	
8		1	1	1	1	1		1

Transitive closure Matrix

---. Explanation for getting the transitive closure matrix & its transpose is not included since we assumed that you would know this.

Partitioning algorithm (cont)



	1	2	3	4	5	6	7	8
1	1							
2		1					1	
3			1	1	1			
4			1	1	1			
5			1	1	1			
6						1		
7		1					1	
8								1

Intersection with its transpose.

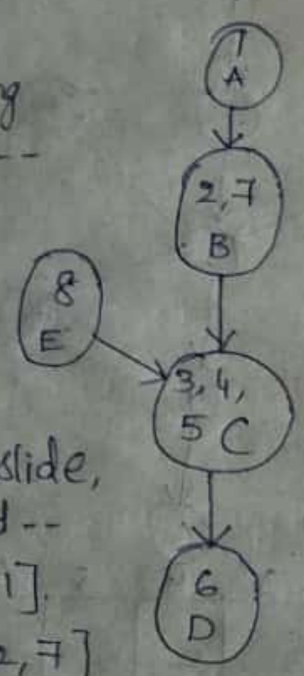
Considering the entires in the above matrix we have the following---

Example: to get values for B - it is evident that B(row 2) has entries at column 2 & 7. Hence the values

A = [1]
B = [2,7]
C = [3,4,5]
D = [6]
E = [8]

Partitioning algorithm (cont...)

The resulting graph is --



From the earlier slide, we had --

A = [1]

B = [2,7]

C = [3,4,5]

D = [6]

E = [8]

	A	B	C	D	E
A	1	1			
B		1	1		
C			1	1	
D				1	
E			1		1

Considering that the graph has self loops Diagonal entries are made to represent self loop.

Building tools (node degree and graph density):

- The out-degree of a node is the number of out-links it has.
- The in-degree of a node is the number of in-links it has
- The degree of a node is the sum of in-degree and out-degree.
- The average degree (mean) of a node is b/w 3 and 4.
- Degree of a simple branch/junction is 3.
- Degree of a loop contained in 1 statement is 4.
- Mean degree of 4 or 5 very busy flow graph.

What's wrong with arrays?

Matrix as a 2-dimensional array is not convenient for larger graphs. Heres why!.. We have four reasons for the same

- Space: For a matrix representation of an array space grows as n^2 whereas, for a linked list, it grows only as kn, where K is a small number such

as 3 or 4.

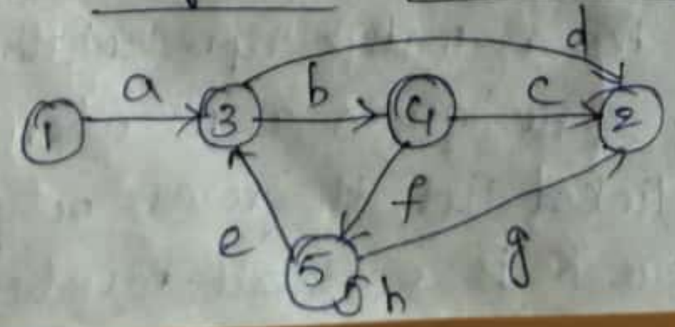
Weights :- Most weights in arrays are complicated and may have many components. This means that an additional weight matrix is required for each such weight.

Variable-length Weights :- If the weights are regular expressions / algebraic expressions, we would then need a 2-dimensional string array (most of whose entries would be null).

Processing time :- Even though operations over null entries are fast, it still takes time to access such entries and discard them.

The matrix representation forces us to spend a lot of time processing combinations of entries that we know will yield null results.

Building tools- Linked list representation



	1	2	3	4	5
1			a		
2					
3		d		b	
4		c			f
5		g	e		h

Every node is a unique name/number

A link is a pair of node names

The linked list entries for the above are ---

node 1,3;a
node 1,2
node 3,2;d
node 3,4;b
node 4,2;c
node 4,5;f
node 5,2;g
node 5,3;e
node 5,5;h

format for linked list entries
row, column, data entry

The link names will usually be pointers to entries in a string array (where actual weight expressions are stored).

If the weights are fixed length, they can be associated with links in parallel fixed entry length array.

Building tools- Linked list representation

Clarifying entries by using node names and pointers.

1,3;a
2,
3,2;d
3,4;b
4,2;c
4,5;f
5,2;g
5,3;e
5,5;h

list entry	Content
1	node 1,3; a
2	node 2, exit
3	node 3,2; d
	3,4;b
4	node 4,2; c
	4,5;f
5	node 5,2; g
	5,3;e
	5,5;h

Fig: 1 is represented in a more logical format in figure 2 by representing the nodes under list entry.

Fig 3 is more detailed the with-in links to that node added.

list entry	content
1	node 1,3;a
2	node 2, exit
	3,
	4,
	5,
3	node 3,2;d
	3,4;b
	1,
	5,
4	node 4,2;c
	4,5;f
	3,
5	node 5,2;g
	3;e
	5;h
	4,
	5

Matrix operations

. parallel Reduction:

The easiest operation. After sorting, Parallel links are are adjacent entries

with the same pair of node names

Ex:- say we have 3 parallel links from node 17 to node 44. Y, Z & W are the pointers to the weight expressions

Depicting all the entires for parallel links between node 17 & node 44, we have

node 17, 21; X
, 44; Y
, 44; Z
, 44; W

node 17, 21; X
, 44; Y

where $Y = y + z + w$

Loop Reduction:- The self loop is identified. To remove the loop, the link weight must be multiplied with all the out links from that node.

Start by identifying the out-links to be multiplied. Multiply the self loop (h^*) where h is the link weight of the self loop with the out-link.

Example:- From the below entries, it is evident at entry (5, 5; h) that it is a self loop at node 5 with link weight h. Also, from the 1st two entries, we see that the out-links from node 5 are 2 and 3. We need to multiply the self loop (h^*) with the link weights of nodes from node 5 to node 2 & 3.

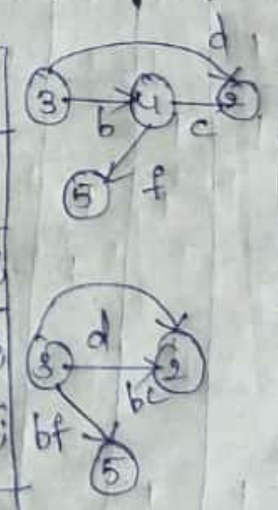
node 5, 2; g
5, 3; e
5, 5; h

node 5, 2; h*g
5, 3; h*e

Cross-Term Reduction:- Select a node for reduction. The cross-term step requires that you combine every in-link to the node with every out-link from that node. The in-links are obtained by back pointers. The new links created be removing the node will be associated with the nodes of the in-links.

Example:- Say that the node to be removed was node 4.

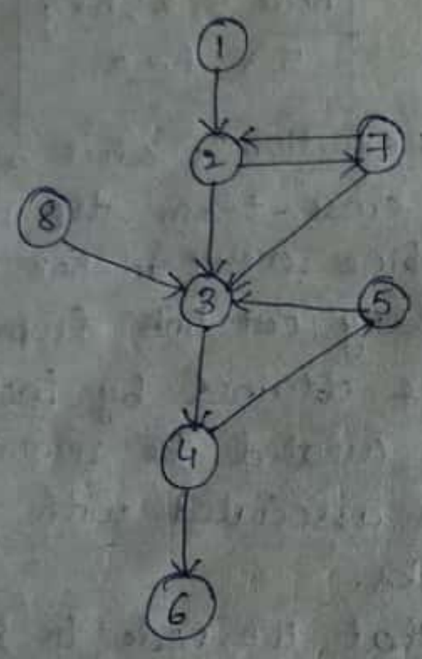
list entry	Content
2	node 2, exit
	(idlink) 4, 2
	(inlink) 3, 2
3	node 3, 2; d
	3, 4; b
4	node 4, 2; c
	4, 5; f
	(inlink) 3, 4; b
5	(inlink) 4, 5



list entry	Content
2	node 2, exit
	(inlink) 3, 2
3	node 3, 2; d
	3, 2; bc
	3, 5; bf
5	(inlink) 4, 5

Partitioning algorim

Consider the following flowgraph and its graph matrix (A+I)



A+I =

	1	2	3	4	5	6	7	8
1	1	1						
2		1	1				1	
3			1	1				
4				1	1	1		
5			1		1			
6						1	1	
7		1	1					
8			1					

The transitive closure matrix $(A+I)^n$ can be obtained by using the following steps.

step:1 : Mark all diagonal entries by 1

step 2:- The flow from node 1 to node 6 is 1-2-7-2-3-4-5-3-4-6

So mark nodes 1,2,3,4,5,6,7 by 1 in the first row

step 3:- The flow from node 2 to node 6 is 2-7-2-3-4-5-3-4-6

so mark nodes 2,3,4,5,6 by 1 in the second row.

step 4:- The flow from node 3 to node 6 is 3-4-5-3-4-6

so mark nodes 3,4,5,6 by 1 in the third row.

step 5:- The flow from node 4 to node 6 is 4-5-3-4-6

so mark nodes 3,4,5,6 by 1 in the fourth row.

step 6:- The flow from node 5 to node 6 is 5-3-4-6

so mark nodes 3,4,5,6 by 1 in the fifth row.

step 7:- The flow from node 6 is only 6.

so mark node 6 by 1 in the sixth row.

step 8:- The flow from node 7 to node 6 is 7-2-3-4-5-3-4-6
so mark nodes 2, 3, 4, 5, 6, 7 by 1 in the seventh row.

step 9:- The flow from node 8 to node 6 is 8-3-4-5-3-4-6
so mark nodes 3, 4, 5, 6, 8 by 1 in the eigth row

Therefore the transitive closure matrix is

$(A+I)^n =$

	1	2	3	4	5	6	7	8
1	1	1	1	1	1	1	1	
2		1	1	1	1	1	1	
3			1	1	1	1		
4			1	1	1	1		
5			1	1	1	1		
6						1		
7		1	1	1	1	1	1	
8			1	1	1	1		1

The transpose of $(A+I)^n$ is

$(A+I)^{nT} =$

	1	2	3	4	5	6	7	8
1	1							
2	1	1					1	
3	1	1	1	1	1		1	1
4	1	1	1	1	1		1	1
5	1	1	1	1	1		1	1
6	1	1	1	1	1	1	1	1
7	1	1					1	
8								1

The intersection of transitive closure matrix $(A+I)^n$ and transpose matrix $(A+I)^{nT}$ is given by identifying similar rows and column entries from $(A+I)^n$ and $(A+I)^{nT}$

$(A+I)^n \# (A+I)^{nT} =$

	1	2	3	4	5	6	7	8
1	1							
2		1					1	
3			1	1	1			
4			1	1	1			
5			1	1	1			
6						1		
7		1					1	
8								1

From the above matrix, by comparing a row/column with other rows/columns the equivalent nodes to be grouped.

After grouping

(repeated from the above matrix, by comparing a row/column with other rows/columns, the equivalent nodes to be grouped.) repeated

Let A = [1] B = [2,7]

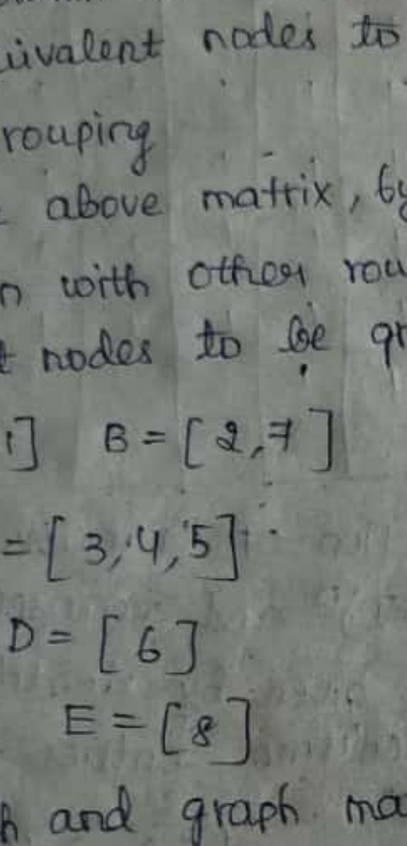
C = [3,4,5]

D = [6]

E = [8]

The graph and graph matrix representation to the above values is given by

Flow Graph



Graph Matrix

	A	B	C	D	E
A	1	1			
B		1	1		
C			1	1	
D				1	
E			1		1

18/3/26

Building tools

The building tools of a graph matrix is a combination of several things such as

1) matrix representation in software
2) Matrix operation
3) Node Reduction optimization.

Matrix Representation in software

→ The graphs are the figures that are used to represent the flow of actions in a system to study the behaviour of the system

→ The degree of nodes is represented as the sum of indegree and outdegree

→ The indegree is the number of inner links of the particular node

→ The outer degree is the number of out links from the particular node

Advantages of graph matrix

① Analazing the ~~routes~~ routes.
② gives best result
③ For proving and developing algorithms

Disadvantages of graph matrix

① Large storage space, weight matrix ②
③ Time duration.

Matrix operations

parallel link reduction

It is the simplest of all the matrix operations.

serial link reduction (or) cross term reduction :-

means the reduction is one of the particular operation of all the availa matrix and involves removal of the node.

A node is first selected for removal and is then removed by concating all its inner and outer link weights

Reduction of loops :-

It means we can remove off the loop

→ Loops can be identified by analazing the linked list of the nodes

→ self link represented the loop term.

Matrix powers and products

. Given a matrix whose entries are a_{ij}, the square of that matrix is obtained by replacing every entry with

$$a_{ij} = \sum_{k=1}^{n} a_{ik} a_{kj}$$

. More generally, given two matrices A and B with entries b_{kj}, respectively, their product is a new matrix C, whose entries are c_{ij}, where:

$$c_{ij} = \sum_{k=1}^{n} a_{ik} b_{kj}$$

3.1 The set of all paths

Our main objective is to use matrix operations to obtain the set of all paths between all nodes or, equivalently, a property (described by link weights) over the set of all paths from every node to every other node, using the appropriate arithmetic rules for such weights. The set of all paths between all nodes

is easily expressed in terms of matrix operations. It's given by the following infinite series of matrix powers.

This is an eloquent, but practically useless, expression. Let I be an n by n matrix, where n is the number of nodes. Let I's entries consist of multiplicative identity for those weights. The above product can be re-phrased as:

$$A(I + A + A^2 + A^3 + A^4 \cdots A^\infty)$$

But often for relations, $A + A = A \cdot (A + I)$

$$= A^2 + A + A + IA^2 + A + I.$$

Furthermore, for any finite n,

$$(A+I)^n = I + A + A^2 + A^3 \cdots A^n$$

Therefore, the original infinite sum can be replaced by

$$\sum_{i=1}^{\infty} A^i = (A+I)^\infty$$

This an improvement, because in the original expression we had both infinite products and infinite sums and now we have only one finite product to contend with. The above is valid wheather or not there are loops. If we restrict our interest for the moment to paths of length n-1, where n is the number of nodes the set of all such paths is given by

$$\sum_{i=1}^{n-1} A^i = A(A+I)^{n-2}$$

This is an interesting set of paths because, with n nodes, no path can exceed n-1 nodes without incorporating some path segment that is already incorporated in some other path or path segment. Finding the set of all such paths is somewhat easier because it is not necessary to do all the intermediate.